



IIC3745: Testing
Actividad 2: Mutation Testing con LLMs
Juan Pablo Sandoval

Miércoles 26 de agosto de 2026

1 Objetivos

La Actividad 1 midió **cobertura**. Esta actividad mide algo distinto: si los tests, además de ejecutar el código, son capaces de **detectar fallas**.

Al finalizar, se espera que puedas:

- Explicar por qué el **mutation score** es una métrica más exigente que la cobertura, y en qué se diferencian.
- Clasificar mutantes en **killed**, **sobrevivientes**, **inválidos** y **equivalentes**, y justificar cada clasificación.
- Reconocer que un mutation score alto puede deberse a una batería sólida o a mutantes triviales, y distinguir ambos casos.
- Evaluar si un LLM propone mutantes más informativos que un generador mecánico de operadores.

2 Descripción

Trabajas sobre el mismo `ReservationService` de la Actividad 1, acompañado de una **batería base de 47 tests** que ya alcanza **100% de statement y branch coverage**. Esa batería está dada y no se modifica.

Debes pedirle a **dos modelos distintos** (GPT y Gemini) que generen **20 mutantes cada uno**, evaluarlos contra la batería base, y comparar ambos resultados contra un **baseline de mutantes clásicos** que genera el notebook.

No debes modificar el código en `src/` ni la batería en `tests/test_base.py`.

2.1 Qué es un mutante

Un mutante es una modificación pequeña a **una sola línea** del código bajo prueba, que altera su comportamiento. La idea es simular una falla que un programador podría haber introducido, y preguntarse si la batería de tests la habría detectado.

Cada mutante se describe con tres campos: la **línea** que modifica, el **precode** (la línea original, copiada carácter por carácter) y el **aftercode** (la línea modificada).

2.2 Clasificación de los mutantes

Estado	Qué significa
killed	Al menos un test de la batería base falla. El mutante fue detectado.
sobrevive	Todos los tests pasan. La batería no detecta ese cambio.
invalido	No se pudo aplicar: la línea no calza con el <i>precode</i> , el resultado no compila, la línea está fuera de rango, o el <i>aftercode</i> es idéntico al original.
equivalente	Sobrevive y tú argumentas que es semánticamente idéntico al original.

El **mutation score ajustado** se calcula como:

$$\text{score} = \frac{\text{killed}}{\text{total} - \text{inválidos} - \text{equivalentes}}$$

Descontar inválidos y equivalentes es lo que hace que el número signifique algo. Un mutante inválido nunca se ejecutó, y un mutante equivalente no puede ser matado por ningún test: dejarlos en el denominador castigaría a la batería por razones que no tienen que ver con su calidad.

2.3 El prompt

La celda `mostrar_prompt(20)` del notebook arma el mensaje completo: la instrucción en inglés seguida del código con números de línea explícitos.

Recomendación 1. La numeración no es decorativa. Sin ella el modelo cuenta mal las líneas y la mayoría de sus mutantes salen con la línea corrida en uno, lo que los vuelve inválidos antes de poder evaluarlos. No edites el prompt ni quites la numeración.

El modelo debe responder con un array JSON como el siguiente, sin explicaciones ni bloques de markdown alrededor:

```
[
  { "id": 1, "line": 87, "precode": "    if days_ahead >= 30:",
    "aftercode": "    if days_ahead > 30:" },
  ...
]
```

Guardas ese array en `mutantes/gpt.json` y `mutantes/gemini.json`, y la conversación completa en `prompts/gpt.md` y `prompts/gemini.md`.

2.4 Reglas del experimento

1. **Mismo prompt para ambos modelos.** Lo que se compara es el modelo, no la instrucción. Usa el que genera el notebook, sin modificarlo.
2. **20 mutantes por modelo.** Si el modelo entrega menos, repórtalo tal cual; no completes la lista a mano.

3. **Una sola solicitud por modelo.** No iteres ni pidas correcciones. Se evalúa lo que entrega en el primer intento.
4. **Un chat nuevo por modelo.** Y por la misma razón que en la Actividad 1: no se aceptan agentes con acceso al repositorio (Cursor, Copilot Agent, Claude Code), solo la ventana de chat.
5. **No arregles los mutantes inválidos.** Si el modelo copió mal una línea o propuso algo que no compila, eso es un resultado. La tasa de mutantes inválidos es uno de los datos que compara a los dos modelos.
6. **Evidencia completa.** El prompt y la respuesta completa van en `prompts/`. No se aceptan enlaces de tipo “compartir chat”.

Recomendación 2. Los LLMs no son deterministas: dos personas con el mismo prompt obtendrán listas de mutantes distintas. Eso es esperado y no afecta tu nota. El baseline clásico, en cambio, **sí** es determinista y debe dar el mismo número a todo el curso; si el tuyo difiere, es señal de que `src/` o la batería fueron modificados. Se aplicarán controles de similitud sobre `mutantes/` y `prompts/`.

2.5 Mutantes equivalentes

Un mutante equivalente cambia el texto del programa pero no su comportamiento observable: ninguna entrada posible produce una salida distinta. Por definición, ningún test puede matarlo.

Para cada sobreviviente debes decidir si es equivalente o si revela un hueco real en la batería. Si sostienes que es equivalente, agrega dos campos a ese mutante en el JSON y vuelve a ejecutar la celda de evaluación:

```
{ "id": 7, "line": 88, "precode": "          rate = 0.0",  
  "aftercode": "          rate = 0",  
  "equivalente": true,  
  "justificacion": "0.0 y 0 son el mismo valor; rate solo se usa en una multiplicacion" }
```

Recomendación 3. Marcar como equivalente un mutante que en realidad revela un hueco en la batería es el error más común de esta actividad, y sube artificialmente el score. Un equivalente sin justificación no recibe puntaje, y una justificación incorrecta se penaliza igual que un sobreviviente no analizado.

2.6 Baseline: mutantes clásicos

La función `generar_clasicos()` produce mutantes con operadores estándar de mutation testing (reemplazo de operadores relacionales, aritméticos y lógicos, y reemplazo de constantes) directamente sobre el código. Los emite en el mismo formato, así que el mismo evaluador los procesa.

Este es tu punto de comparación. La pregunta de fondo es si los mutantes que propone un LLM son *más difíciles de matar* que los mecánicos, o si simplemente son variaciones que la batería ya cubría.

2.7 Cuando una categoría queda vacía

Es perfectamente posible que alguna categoría te quede en cero. **Eso no es un problema y no te resta puntaje**, siempre que lo dejes dicho explícitamente en el reporte en vez de omitir la sección.

- **Si un modelo no produjo mutantes inválidos:** escribe “0 mutantes inválidos” y sigue. Es un resultado a favor de ese modelo, y de hecho es material para la pregunta 1 del análisis.
- **Si no identificaste mutantes equivalentes:** no inventes ninguno. Basta con señalar que no hubo, y explicar en una línea por qué cada sobreviviente es un hueco real de la batería y no un cambio semánticamente neutro. Recuerda que marcar como equivalente algo que sí es un hueco se penaliza.
- **Si todos los mutantes de un modelo murieron (score 100%):** no hay nada que agregar. Lo que sí debes hacer es discutirlo: un 100% con 20 mutantes normalmente indica que el modelo propuso cambios triviales, no que la batería sea perfecta. Compáralo con el score del baseline clásico y explica la diferencia.
- **Sobrevivientes para comentar:** el baseline clásico siempre deja sobrevivientes, así que los tres que pide el reporte siempre están disponibles. Si los mutantes de los modelos murieron todos, toma los tres del baseline clásico y dilo.

Recomendación 4. Esta actividad no te pide subir el mutation score.

La batería `tests/test_base.py` no se modifica. La pregunta 6 del análisis es hipotética: describes qué tests agregarías y por qué, sin escribirlos ni ejecutarlos.

3 Código Base

Canvas. La actividad se encuentra subida en forma de `.zip` y tiene la siguiente estructura:

```
src/models.py
src/reservation.py          # código bajo prueba, NO modificar
tests/__init__.py
tests/test_base.py          # batería de 47 tests, 100% coverage, NO modificar
mutantes/gpt.json           # aquí pegas los 20 mutantes de GPT
mutantes/gemini.json        # aquí pegas los 20 mutantes de Gemini
prompts/gpt.md              # prompt + respuesta completa
prompts/gemini.md
REPORTE.md
Actividad2.ipynb
.coveragerc
```

Verificación previa. La sección 0 del notebook comprueba que la batería base pasa y cubre el 100%. **Ejecútala antes que cualquier otra cosa.** Si la batería no pasara, todos los mutantes quedarían marcados como killed por un test que ya venía fallando, y el score sería 100% sin significar nada.

Ejecución. El notebook nunca modifica `src/reservation.py` en disco: cada mutante se compila en memoria y se inyecta al momento de correr la batería. Las celdas selladas no deben modificarse.

4 Reporte de Resultados

En `REPORTE.md` debes completar la siguiente tabla. El notebook genera estos valores, pero eres responsable de que la tabla sea consistente con lo entregado.

Fuente	Total	Inválidos	Válidos	Equiv.	Killed	Sobreviven	Score
gpt							
gemini							
clasico							

Además debes entregar, en `REPORTE.md`:

- La **clasificación de los mutantes inválidos** de cada modelo, indicando el motivo (línea corrida, *precode* mal copiado, no compila, no cambia nada).
- La **lista de equivalentes** con su justificación.
- **Tres sobrevivientes no equivalentes** comentados en detalle: por qué la batería no los detecta y qué test los mataría (nombre, entrada y aserción). Pueden venir de cualquiera de las tres fuentes.

4.1 Análisis

Cada respuesta debe apoyarse en un número de tu tabla o en un mutante específico, citado por su identificador.

1. ¿Qué modelo produjo más mutantes inválidos? ¿De qué tipo fueron los errores?
2. Compara los scores de GPT y Gemini. Si uno es más alto, ¿significa que ese modelo generó mutantes *mejores*? Explica por qué un score alto puede indicar mutantes triviales en vez de una batería sólida.
3. La batería base tiene 100% de statement y branch coverage. ¿Qué mutation score obtuvo contra los mutantes clásicos? Explica con un ejemplo concreto por qué la cobertura no garantiza detección de fallas.
4. ¿Los mutantes del LLM fueron más difíciles de matar que los clásicos? Compara los tres scores y da un ejemplo de un mutante que solo una de las fuentes propuso.
5. ¿Qué tipo de mutante sobrevivió con más frecuencia? ¿Qué te dice eso sobre lo que la batería base no está verificando?
6. Si tuvieras que agregar tres tests a la batería base para subir el mutation score, ¿cuáles serían y por qué esos?

5 Evaluación

Igual que en la Actividad 1, **la nota no depende del score que obtengan tus mutantes**, sino del rigor con que ejecutes y documents el experimento.

Criterio	Puntaje
Los dos modelos están documentados en <code>prompts/</code> con el prompt y la respuesta completa, usando el prompt generado por el notebook	1,0
<code>mutantes/</code> contiene las dos tandas tal como las entregaron los modelos, sin correcciones manuales	0,5
La tabla de resultados está completa y es consistente con lo entregado	1,0
Los mutantes inválidos están clasificados por motivo, o se indica que no hubo	0,5
Los mutantes equivalentes están correctamente identificados y justificados; si no hubo, se señala y se argumenta por qué cada sobreviviente es un hueco real	1,0
Tres sobrevivientes, de cualquiera de las tres fuentes, están comentados con el test que los mataría	1,5
Las seis preguntas de análisis están respondidas con evidencia concreta	1,5
Total	7,0

Recomendación 5. “El mutante sobrevive porque el test no lo detecta” no es una respuesta. Para cada sobreviviente se espera que identifiques *qué condición del código queda sin verificar* y que escribas el test concreto que cerraría ese hueco.

6 Entregable, Fecha de Entrega y Penalizaciones

Debes renombrar el archivo `Actividad2.ipynb` con tu número de alumno/a (por ejemplo, `17626465.ipynb`) y entregar:

- el notebook renombrado,
- los dos archivos de `mutantes/`,
- los dos archivos de evidencia en `prompts/`, y
- el archivo `REPORTE.md` completo.

Adicionalmente, debes comprimir las carpetas `mutantes/` y `prompts/` en un único archivo `numeroAlumno.zip` y subirlo junto con la entrega.

6.1 Fechas de Entrega y Penalizaciones

- **Fecha de entrega:** Miércoles 2 de septiembre hasta las 23:59.
- **Atraso menor a 1 hora (-0,5 puntos):** Jueves 3 de septiembre desde 00:00 hasta las 00:58.
- **Atraso de 24 horas (-1 punto):** Jueves 3 de septiembre desde 00:59 hasta las 23:59.
- **Atraso de 48 horas (-2 puntos):** Viernes 4 de septiembre desde 00:00 hasta las 23:59.

Nota: Los descuentos por atraso se aplican a la nota final de la actividad.